

Modelos clásicos: regresión, árboles y SVM

Módulo 3 · Machine Learning Clásico

Regresión logística, árboles de decisión, Random Forest, XGBoost y LightGBM: el arsenal del ML tabular en producción empresarial.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Modelos clásicos: regresión, árboles y SVM

Los modelos clásicos de ML son el caballo de batalla del 80% de los proyectos de datos en empresa. Regresión logística, árboles de decisión, Random Forest, XGBoost: estos algoritmos resuelven de forma efectiva y eficiente la mayoría de los problemas de predicción sobre datos estructurados. Son interpretables (o al menos interpretables con herramientas como SHAP), baratos de entrenar, rápidos de inferir, y maduros en producción. Antes de plantear cualquier solución con LLMs o deep learning sobre datos tabulares, estos modelos deben ser el baseline obligatorio.

XGBoost y LightGBM dominan consistentemente los benchmarks de datos tabulares y las competiciones de Kaggle con datos estructurados. Un paper de revisión de IBM Research (arXiv:1809.04559) que comparó XGBoost, LightGBM y CatBoost en cuatro datasets a gran escala encontró que todos son competitivos, con XGBoost y LightGBM consistentemente en el top en términos de precisión y velocidad. Para sectores regulados, la regresión logística sigue siendo el modelo de referencia por su interpretabilidad perfecta y sus propiedades matemáticas bien entendidas.

Los modelos fundamentales y su lógica

Regresión lineal y logística: la base interpretable

La regresión lineal predice un valor continuo como combinación lineal de las variables de entrada: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$. Los coeficientes β tienen interpretación directa: "por cada unidad de aumento en x_i , y aumenta en β_i unidades, manteniendo las demás variables constantes". Esta interpretabilidad perfecta es su ventaja principal en contextos donde hay que explicar las predicciones a no técnicos o a reguladores.

La regresión logística aplica una función sigmoide sobre la combinación lineal para producir una probabilidad entre 0 y 1: $P(y=1|x) = 1/(1+e^{-(\beta \cdot x)})$. Es el modelo estándar para clasificación binaria en sectores regulados. FICO Score, muchos modelos de elegibilidad crediticia y los modelos de riesgo de seguros son regresiones logísticas o variantes próximas, precisamente porque sus coeficientes son directamente auditables y su comportamiento es matemáticamente predecible.

Árboles de decisión: reglas jerárquicas aprendidas de los datos

Un árbol de decisión divide iterativamente el espacio de variables en regiones cada vez más homogéneas respecto al target. Cada nodo interno aplica una condición ($x_i > 5.3?$), y según la respuesta dirige el ejemplo hacia la rama izquierda o derecha, hasta llegar a una hoja que produce la predicción. Los árboles son completamente interpretables (se pueden visualizar y leer), no requieren normalización de los datos, manejan variables categóricas de forma nativa, y capturan automáticamente interacciones entre variables.

Su limitación es la varianza alta: pequeños cambios en los datos de entrenamiento pueden producir árboles muy distintos. Esto se resuelve con los métodos de ensemble.

Random Forest y Gradient Boosting: potencia de los ensembles

Random Forest entrena muchos árboles de forma independiente sobre muestras aleatorias de los datos y características, y combina sus predicciones por votación o promedio. El promedio de muchos árboles con alta varianza produce un modelo con varianza mucho menor, manteniendo el bias bajo. El resultado es un modelo robusto que generaliza bien incluso con pocas tunificaciones.

Gradient Boosting (XGBoost, LightGBM, CatBoost) construye los árboles de forma secuencial: cada árbol nuevo corrige los errores del ensemble anterior, ajustándose a los residuos (los errores que el modelo actual comete). El resultado es un modelo de alta precisión que suele superar a Random Forest en benchmarks tabulares a cambio de más hiperparámetros a ajustar y algo más de tiempo de entrenamiento.

SVM: máquinas de vectores soporte

Las SVM encuentran el hiperplano que maximiza el margen entre las clases. Con el kernel trick, pueden aplicar esta idea en espacios de alta dimensionalidad sin computar explícitamente las coordenadas en ese espacio, permitiendo separar clases no linealmente separables en el espacio original. Fueron el estado del arte en clasificación de texto antes de los transformers y siguen siendo útiles para datasets de tamaño medio con pocas características relevantes.

SECCIÓN 3 · CÓMO FUNCIONA

XGBoost y LightGBM: el estado del arte en datos tabulares

XGBoost añade a Gradient Boosting estándar: regularización L1 y L2 en el objetivo (evita sobreajuste), procesamiento paralelo para entrenamiento rápido, manejo nativo de valores nulos (aprende automáticamente la mejor dirección para los datos faltantes), y soporte para GPU. LightGBM usa histogramas para aproximar los splits óptimos (lo que lo hace mucho más rápido en datasets grandes) y crecimiento leaf-wise en lugar de level-wise (produce árboles más profundos y precisos pero más propensos a sobreajustar sin regularización adecuada). CatBoost destaca en el manejo nativo de variables categóricas sin one-hot encoding.

Los hiperparámetros más importantes para gradient boosting: `n_estimators` (número de árboles, más es mejor si hay regularización), `learning_rate` (0.01-0.1 es el rango típico, valores menores requieren más árboles), `max_depth` (3-8 es el rango típico, más profundidad = más complejidad = más riesgo de sobreajuste), `subsample` (porcentaje de datos por árbol, 0.7-0.9, reduce varianza), y los parámetros de regularización (`reg_alpha`, `reg_lambda`). Un tuning básico con optuna o hyperopt sobre estos parámetros puede mejorar el AUC-ROC un 2-5% respecto a los valores por defecto.

SECCIÓN 4 · EJEMPLOS REALES

- Scoring de crédito con regresión logística (BBVA, Santander): los modelos de scoring regulados en Europa usan regresión logística o variantes cercanas porque el GDPR y las

regulaciones bancarias exigen explicabilidad de las decisiones crediticias. El coeficiente de "ratio_deuda_ingresos" en la regresión dice exactamente cuánto pesa esa variable en la decisión.

- Detección de fraude con XGBoost (PayPal, Stripe): los sistemas de detección de fraude en pagos procesan millones de transacciones por segundo. XGBoost con menos de 1ms de latencia de inferencia y AUC-ROC de 0.99+ es el estándar de la industria para este problema.
- Predicción de demanda con LightGBM (Walmart, Carrefour): predicción de demanda por producto-tienda-semana sobre datasets con millones de filas. LightGBM es la elección por su velocidad en datasets grandes y su buen manejo de variables categóricas (tienda, producto, categoría) sin preprocessing.
- Churn prediction con Random Forest (Telefónica, Vodafone): modelos de predicción de abandono de clientes con cientos de variables (actividad, uso de servicios, historial de reclamaciones, datos demográficos). Random Forest produce feature importances que permiten identificar los principales drivers de churn para las acciones de retención.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar deep learning o LLMs para datos tabulares sin comparar con XGBoost. En benchmarks sistemáticos (Grinsztajn et al., 2022, NeurIPS), XGBoost y LightGBM superan a las redes neuronales en la mayoría de los datasets tabulares medianos. Para datos estructurados, XGBoost es el baseline que hay que superar antes de justificar la complejidad de una red neuronal.

Error 2: No hacer feature engineering básico antes de entrenar. Los modelos de gradient boosting aprenden bien las interacciones entre variables, pero algunas transformaciones básicas (log de variables con distribución sesgada, ratios entre variables relacionadas, fechas descompuestas en día/mes/hora) pueden mejorar el rendimiento significativamente sin añadir complejidad al modelo.

Error 3: Usar feature importance de XGBoost directamente para explicabilidad. La feature importance nativa de XGBoost (número de veces que una variable aparece en un split) es inestable y puede ser engañosa en presencia de variables correlacionadas. SHAP values son significativamente más fiables para explicabilidad y deben preferirse.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El pipeline mínimo viable para un proyecto de ML supervisado con datos tabulares: carga los datos con pandas, analiza la distribución de variables y valores nulos con `df.describe()` y `df.isnull().sum()`, aplica preprocessing básico (imputación de nulos, encoding de categóricas), divide en train/test con `train_test_split`, entrena un modelo baseline de regresión logística o Random Forest, evalúa con las métricas apropiadas (AUC-ROC para clasificación desbalanceada, RMSE para regresión), compara con XGBoost, y aplica SHAP para entender qué variables son más relevantes.

Para el tuning de hiperparámetros en producción, Optuna es el framework estándar: define el espacio de búsqueda (rangos de los hiperparámetros), el objetivo (maximizar AUC-ROC en validación) y el número de trials (50-200 es suficiente para la mayoría de los casos). Optuna usa

Bayesian optimization (TPE) para explorar el espacio de hiperparámetros de forma mucho más eficiente que grid search.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M2-T4 (Funciones de coste): XGBoost acepta funciones de coste personalizadas que permiten alinear la optimización con la métrica de negocio real.
- M3-T5 (Evaluación): AUC-ROC, precisión, recall y F1 son las métricas estándar para evaluar los modelos de este tema.
- M3-T7 (Interpretabilidad): SHAP values para XGBoost son la herramienta de interpretabilidad más usada en modelos tabulares en producción.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los modelos clásicos de ML para datos tabulares son el estándar en producción empresarial: regresión logística para clasificación interpretable en sectores regulados, árboles de decisión para reglas visualizables, Random Forest para robustez con pocas tunificaciones, y XGBoost/LightGBM para máxima precisión. XGBoost y LightGBM son la referencia en benchmarks tabulares y deben ser el baseline de cualquier proyecto antes de escalar a deep learning. Feature importance nativa es inestable; usar SHAP values para explicabilidad. Protocolo: baseline logístico → XGBoost default → XGBoost tuned → solo entonces deep learning.